Real-World Applications of RSA: Cryptographic Foundations, Practical Implementation, and Off-Chain Blockchain Authentication

Academic Research Paper in Cryptography, Cybersecurity, and Blockchain Systems

Abstract— The RSA cryptosystem remains a foundational pillar of modern public-key cryptography. While public blockchain consensus protocols predominantly utilize elliptic-curve cryptography (ECC) for on-chain state compactness, RSA plays an enduring role in enterprise infrastructure, Public Key Infrastructure (PKI), and internal gateway authorization. This report provides an academic investigation of real-world RSA applications across theory, operational justification, and systems engineering. We formulate RSA's mathematical foundations, contrasting RSA-OAEP key transport with RSA-PSS digital signatures (WHAT). We analyze why RSA is favored in specific trust architectures, providing an evidence-based comparison against ECDSA and Schnorr signatures (WHY). Finally, we design, implement, and benchmark an educational off-chain cryptocurrency withdrawal authentication system using RSA-PSS and SHA-256 (HOW). Empirical benchmarks across RSA (2048, 3072, 4096 bits) and ECDSA (secp256k1) on identical hardware demonstrate RSA's distinct operational profile: while RSA signing is slower than ECDSA, RSA public exponent verification executes significantly faster across all tested key sizes—RSA-2048 is 13.45x faster, RSA-3072 is 8.22x faster, and RSA-4096 is 5.4x faster than ECDSA verification. At the approximately equivalent 128-bit classical security level (RSA-3072 vs. ECDSA-secp256k1), RSA-3072 verification is 8.22x faster than ECDSA, whereas ECDSA signing is 3.87x faster than RSA-3072. Adversarial security testing validates that combining cryptographic digital signatures with stateful timestamp freshness and nonce replay caches significantly strengthens request authentication against payload tampering and replay attacks. Evaluation against a bundled RSA-PSS 2048-bit SHA-256/MGF1-SHA256 Wycheproof test set (version 0.9 snapshot, 108 test vectors) confirms that the verification engine correctly accepts all 63 expected-valid test cases and rejects all 45 expected-invalid test cases, demonstrating agreement with this test set without establishing complete protocol certification.

Keywords: *RSA Cryptography, RSA-PSS, Digital Signatures, PKI, Hybrid Encryption, Cryptocurrency Security, Off-chain Authentication, Empirical Evaluation.*

1. Introduction and Research Scope

Modern information assurance rests on four foundational security objectives: confidentiality, data integrity, entity authenticity, and non-repudiation evidence [1], [2]. Under Kerckhoffs's principle, the security of a cryptosystem must depend solely on the secrecy of the private key, rather than on the obscurity of the underlying algorithm. First documented by Ron Rivest, Adi Shamir, and Leonard Adleman in their April 1977 MIT research memorandum and published in Communications of the ACM in 1978 [1], and standardized under PKCS #1 / RFC 8017 [2], the RSA cryptosystem helped address the historic key distribution challenge by separating the private transformation used for decryption and signing from the public transformation used for encryption and verification. Although asymmetric cryptography eliminates the need to transmit secret keys over insecure channels, it does not remove the necessity for authentic public-key distribution, requiring Public Key Infrastructure (PKI), certificate authorities, or trusted directories to prevent impersonation and man-in-the-middle attacks. For nearly five decades, RSA has underpinned digital commerce, software distribution, and enterprise identity management.

In modern blockchain environments, significant architectural debate centers on algorithm selection. Major public networks—including Bitcoin and Ethereum—rely on Elliptic Curve Cryptography (ECC) for native on-chain transaction signatures [8], [12], [13]. In enterprise cryptocurrency operations, transaction handling does not occur solely on-chain; financial backends employ layered off-chain microservices, Hardware Security Modules (HSMs), and Transport Layer Security (TLS 1.3) tunnels [3] to authenticate and authorize requests before dispatching them to blockchain nodes. This report investigates real-world RSA applications across three dimensions: (1) WHAT: mathematical principles and operational distinctions between RSA-OAEP key transport and RSA-PSS signatures; (2) WHY: security justification of RSA and an evidence-based comparison with ECDSA and Schnorr; (3) HOW: practical design, implementation, and empirical evaluation of an educational off-chain cryptocurrency withdrawal authentication system using RSA-PSS.

2. RSA Fundamentals and Real-World Applications

2.1 Mathematical Foundations and Operating Mechanics

The security of RSA rests on the computational intractability of the integer factorization problem and the presumed hardness of the RSA problem [1], [2]. Given modulus $n = p \cdot q$ (where p and q are large distinct prime numbers), factoring n immediately reveals Euler's totient $\varphi(n) = (p - 1)(q - 1)$, enabling the derivation of private exponent $d \equiv e^{-1} \pmod{\varphi(n)}$ via the Extended Euclidean Algorithm solving Bézout's identity $e \cdot d + k \cdot \varphi(n) = 1$. The RSA problem asks to compute plaintext m given ciphertext $c \equiv m^e \pmod{n}$, public exponent e, and modulus n without knowing the prime factors. While an efficient factoring algorithm immediately breaks RSA, whether the RSA inversion problem is strictly equivalent to integer factorization remains an unproven hypothesis in complexity theory. In practical operations: (i) public exponent e is chosen coprime to $\varphi(n)$, commonly chosen as 65,537, which is a Fermat prime ($e = 2^{16} + 1$); (ii) private exponent d satisfies $e \cdot d \equiv 1 \pmod{\varphi(n)}$; and (iii) Chinese Remainder Theorem (CRT) parameters (p, q, dP, dQ, qInv) are retained to accelerate private-key operations, where $dP = d \bmod (p - 1)$, $dQ = d \bmod (q - 1)$, and $qInv = q^{-1} \bmod p$, yielding an approximate 4x computational speedup. By Euler's theorem, for any message m coprime to n, $m^{(e \cdot d)} = m^{(k \cdot \varphi(n) + 1)} \equiv m \pmod{n}$. Textbook RSA operates directly on integers ($c = m^e \bmod n$; $m = c^d \bmod n$), but exhibits dangerous algebraic multiplicative homomorphism: $(m_1)^e \cdot (m_2)^e \equiv (m_1 \cdot m_2)^e \pmod{n}$. This allows adversaries to construct valid ciphertexts or forged signature representatives without knowing d. Production deployments mandate modern randomized padding schemes.

2.2 Distinction Between RSA Encryption and RSA Digital Signatures

RSA supports two distinct operational primitives with contrasting security goals:

• Asymmetric Encryption and Key Transport (Confidentiality): RSA-OAEP (RFC 8017 Section 7.1 [2]) provides provable semantic security. In the Random Oracle Model (ROM), Bellare and Rogaway (EUROCRYPT '94, published 1995 [17]) introduced OAEP, proving IND-CPA security and plaintext awareness (implying IND-CCA1 for general trapdoor permutations). Shoup (CRYPTO 2001, Journal of Cryptology 2002 [15]) showed that a general IND-CCA2 proof cannot be established for arbitrary permutations under standard one-wayness, proposing OAEP+. Fujisaki, Okamoto, Pointcheval, and Stern (CRYPTO 2001, Journal of Cryptology 2004 [16]) proved that RSA-OAEP achieves IND-CCA2 security under the standard RSA assumption via partial-domain one-wayness equivalence, though the reduction is non-tight for practical key sizes. Because RSAES-OAEP payload capacity is bounded

by mLen <= k - 2hLen - 2 (RFC 8017 [2]; 190 bytes for RSA-2048 with SHA-256), production systems deploy RSA-OAEP for hybrid key transport (NIST SP 800-56B Rev. 2 [5])—encrypting (wrapping) an ephemeral 256-bit AES key using RSA-OAEP to protect bulk data authenticated with AES-GCM (NIST SP 800-38D [22]), maintaining the distinction between RSA key transport and KEM encapsulation (such as ML-KEM in RFC 10024 [19]).

• Digital Signatures (Authenticity, Integrity, Non-Repudiation Evidence): Under the hash-and-sign paradigm, SHA-256 maps arbitrary-length messages to a fixed-length 256-bit digest, allowing RSA-PSS to operate on a fixed-length digest-derived encoding that fits the RSA modulus rather than applying RSA directly to the full message. However, SHA-256 itself does not eliminate textbook RSA's multiplicative homomorphism. While RFC 8017 [2] and NIST FIPS 186-5 [4] specify the standardized engineering scheme, Bellare and Rogaway [23] established its foundational theoretical security analysis, proving that RSA-PSS achieves an essentially tight EUF-CMA security reduction to the RSA inversion problem in the Random Oracle Model (ROM). In signing, digest mHash = SHA-256(M) and a randomized salt are encoded via EMSA-PSS into octets EM, converted to integer representative m = OS2IP(EM), and signed via private exponentiation s = RSASP1(SK, m) = m^d mod n. During verification, the verifier recovers m = RSAVP1(PK, s) = s^e mod n, formats EM = I2OSP(m, emLen), and executes EMSA-PSS-VERIFY(M, EM, emBits). Modular exponentiation alone does not establish signature validity; full EMSA-PSS verification—unmasking salt via MGF1, checking padding (0xBC trailer), and verifying hash consistency—is indispensable. Signatures provide mathematical non-repudiation evidence, while comprehensive non-repudiation requires identity binding, audit trails, and secure key custody.

• Cryptographic Key Separation (NIST FIPS 186-5 Section 5.1 / SP 800-57): NIST FIPS 186-5 Section 5.1 [4] explicitly mandates cryptographic key separation: an RSA key pair designated for digital signatures shall not be used for key establishment or encryption (such as RSA-OAEP). Maintaining separate key pairs prevents cross-protocol interactions between disparate padding schemes, enforces strict cryptographic purpose separation, minimizes compromise impact, and simplifies compliance auditing. Systems utilizing both signing and encryption must enforce distinct key pairs, separate certificate hierarchies, and independent key-management policies.

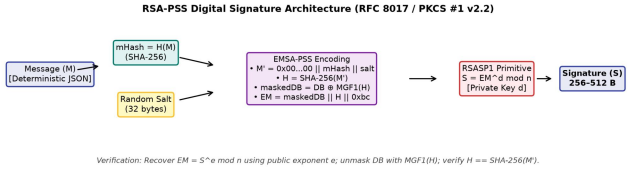


Figure 1. Operational workflow of the RSA-PSS probabilistic digital signature scheme (RFC 8017 / FIPS 186-5).

2.3 Real-World Application Breadth

In industrial systems, RSA is deployed across three primary operational domains: (1) Hybrid Encryption and Key Transport: Rather than encrypting bulk data directly, RSA-OAEP is deployed for key transport—securely encrypting (wrapping) an ephemeral 256-bit AES key used with AES-GCM [22] per NIST SP 800-56B Rev. 2 [5] to combine asymmetric key transport with high-throughput symmetric transmission. (2) Code Signing and Firmware Integrity: Operating systems, application ecosystems, and secure bootloaders widely utilize RSA digital signatures (both RSA-PSS and legacy PKCS#1 v1.5) to verify executable integrity before launch. (3) Digital Certificates and PKI: In TLS 1.3 (RFC 9846 [3], superseding RFC 8446), static RSA key exchange was removed to ensure forward secrecy through ephemeral Diffie-Hellman ((EC)DHE). To prepare for post-quantum threats, RFC 10024 [19] defines hybrid key agreement combining post-quantum mechanisms (ML-KEM) with traditional ECDHE. Consequently, RSA's modern role in TLS 1.3 is focused on endpoint authentication, which distinguishes three separate concepts: (i) an X.509 certificate binds the server's identity to its public key via a Certificate Authority's signature; (ii) the server proves possession of the corresponding private key by signing the handshake transcript within a CertificateVerify message; and (iii) when RSA keys are used for TLS 1.3 server authentication, RFC 9846 [3] strictly requires RSA-PSS for CertificateVerify (the CA's certificate signature is a distinct operation that is not required to be RSA-PSS). RFC 9963 [18] specifies a restricted legacy exception permitting client certificate authentication using RSASSA-PKCS1-v1_5 in constrained environments, but does not permit PKCS#1 v1.5 for server authentication.

3. Why RSA? Security Purpose and Comparative Evaluation

RSA addresses the core security objectives of information assurance: confidentiality (via hybrid key transport), entity authentication, data integrity, and non-repudiation evidence. Table 1 compares RSA with elliptic curve alternatives: ECDSA (secp256k1) [12], [13], Schnorr (BIP 340) [8], and Ed25519 [11], with security strengths benchmarked according to NIST SP 800-56B Rev. 2 (Table 2 and Appendix D) [5] and generic elliptic-curve strength mappings from NIST SP 800-57 Part 1 Rev. 5 [6].

Evaluation Metric	RSA-2048	RSA-3072	ECDSA (secp256k1)	Schnorr (BIP 340)	Ed25519 (RFC 8032)
Hard Problem	Integer Factorization (IFP) / RSA Inversion Problem	Integer Factorization (IFP) / RSA Inversion Problem	EC Discrete Log (ECDLP)	EC Discrete Log (ECDLP)	Edwards EC-DLOG (ECDLP)
Security Strength	112 bits [5]	128 bits [5]	~128 bits (generic EC mapping [6])	128 bits (generic EC mapping [6])	128 bits (generic EC mapping [6])
Key Size / Sig Size	Modulus: 256 B Sig: 256 B DER PubKey: ~294 B	Modulus: 384 B Sig: 384 B DER PubKey: ~422 B	PubKey: 33 B (comp) Bitcoin: 70–72 B (71 B sample) Ethereum: 65 B (r‖s‖v), RLP; 64 B (ERC-2098 [28])	PubKey: 32 B (x-only) BIP 341 [27]: 64 B (DEFAULT) or 65 B (explicit sighash)	PubKey: 32 B Sig: 64 B [11]
Relative Verify Speed	Empirical: 0.046 ms (13.45x vs ECDSA)	Empirical: 0.076 ms (8.22x vs ECDSA)	Empirical: 0.621 ms (Baseline: 1.0x)	Batch verify supported; native compact aggregation (MuSig2 [10])	High efficiency in lit. [11]; not benchmarked

Table 1. Systematic comparison of asymmetric cryptographic algorithms across security and operational metrics.

3.1 Storage and Verification Trade-Offs in Blockchain Consensus Layers

Public blockchain networks prioritize minimizing ledger bloat and transmission overhead, as transaction fees penalize large byte footprints. The mathematical root of this trade-off lies in cryptanalytic complexity: factoring an RSA modulus n = p · q via the

General Number Field Sieve (GNFS) has sub-exponential complexity, whereas solving the Elliptic Curve Discrete Logarithm Problem (ECDLP) in a cyclic group of prime order q via Pollard's rho has expected fully exponential complexity O(sqrt(q)). Consequently, per NIST SP 800-56B Rev. 2 [5] and SP 800-57 Part 1 Rev. 5 [6], RSA requires substantially larger key sizes (e.g., RSA-3072) to match the ~128-bit classical security of a 256-bit curve like secp256k1, inflating signatures and keys. In Bitcoin, fees depend on virtual size (vSize in vB) under BIP 141 (Segregated Witness) [9], where Weight = (Base Size * 3) + Total Size and vSize = ceil(Weight / 4). A standard 1-input, 2-output SegWit P2WPKH transaction has a baseline virtual size of ~141 vB, with a witness stack spanning 107–109 bytes (1-byte count, 1-byte sig len, 71–73 byte DER sig + sighash, 1-byte pubkey len, and 33-byte compressed pubkey) [12]. In a hypothetical model using RSA-2048 for native authorization—under the simplified assumption of a 256-byte modulus payload (treating public exponent e = 65,537 as implicit, rather than complete ~294-byte DER pubkeys) and a 256-byte RSA signature with 3-byte CompactSize prefixes—the witness stack expands to 1 + 3 + 256 + 3 + 256 = 519 bytes. This adds ~410–412 witness bytes per input (~103 additional virtual bytes under BIP 141), increasing virtual size by ~73% (to ~244 vB) and proportionally inflating fees. Native Bitcoin consensus does not support RSA verification; this hypothetical model illustrates why consensus layers mandate compact elliptic curve primitives. Similarly, Ethereum requires 65-byte ECDSA signatures (r, s, v) [13]; while EVM precompile 0x05 (EIP-198) executes raw modular exponentiation, full RSA-PSS verification requires substantial contract bytecode for MGF1 salt unmasking and padding checks. Furthermore, BIP 340 Schnorr signatures [8] integrated via Taproot (BIP 341 [27]) allow 64-byte signatures and enable native compact multi-signature aggregation into a single 64-byte on-chain signature via MuSig2 (BIP 327 [10])—a capability unattainable with standard RSASSA-PSS without specialized multi-party protocols. Similarly, Ethereum smart contracts execute modular exponentiation via precompile 0x05 (EIP-198 [29]) and support compact 64-byte signatures (ERC-2098 [28]), while full RSA-PSS requires substantial EVM bytecode for MGF1 salt unmasking.

3.2 The Operational Value of RSA in Off-Chain Systems

Despite on-chain storage limitations, RSA offers distinct advantages in off-chain exchange backends: (1) Asymmetric Verification Efficiency: RSA verification with common exponent $e = 65{,}537 = 2^{16} + 1$ requires only 17 modular operations (16 squarings and 1 multiplication) under square-and-multiply exponentiation. While modular multiplications and ECC group additions represent distinct operations whose counts cannot be directly equated, the small exponent enables exceptionally fast verification, confirmed by our empirical benchmarks. (2) Enterprise Infrastructure: Banking backends, payment gateways, and HSMs possess mature, hardware-accelerated RSA pipelines. (3) Nonce Reuse Hazards in ECDSA: Standard ECDSA is acutely sensitive to nonce handling. If the same ephemeral nonce k is reused across two distinct message hashes z_1 and z_2 (yielding identical r), private key d_A is algebraically recovered via $s_1 - s_2 \equiv k^{-1}(z_1 - z_2) \pmod n$, revealing $k \equiv (z_1 - z_2)(s_1 - s_2)^{-1} \pmod n$ and $d_A \equiv (s_1 k - z_1) r^{-1} \pmod n$. In contrast, slight nonce bias does not permit this direct two-signature calculation, typically requiring multiple signatures and lattice-based Hidden Number Problem attacks. Because RSA-PSS signature generation does not rely on a secret scalar multiplier, it is fundamentally immune to scalar nonce leakage. However, RSA implementations face distinct side-channel and fault-injection vulnerabilities: (i) Bellcore CRT Fault Attacks: Accelerating RSA signing via the Chinese Remainder Theorem (CRT) is vulnerable to fault injection (Boneh et al. [7]). If a fault corrupts one CRT branch (modulo q while modulo p is intact), faulty signature s' can reveal prime factor p = gcd((s')^e − m, n) if encoded representative m is known. In the two-signature formulation gcd(s − s', n) = p, both valid s and faulty s' must share identical representative m; because RSA-PSS randomizes EM via salt, independent signatures on message M yield $EM_1 \neq EM_2$, preventing this attack across separate runs. (ii) Defense Differentiation: RSA base blinding (randomizing x' = x · r^e mod n) mitigates timing side-channels, but does not prevent CRT fault injection; fault mitigation mandates post-computation verification (checking s^e mod n == m before release) or redundant hardware circuitry.

4. Proposed System Design and Implementation

We design an off-chain architecture: Using RSA-PSS Digital Signatures to Authenticate Off-Chain Withdrawal Requests in Cryptocurrency Exchanges. Architectural Boundary: The proposed architecture operates strictly off-chain within an internal exchange authorization boundary to authenticate withdrawal requests before they reach hot-wallet signers; it does not replace on-chain consensus signatures, nor does it represent the proprietary architecture of any specific commercial platform.

4.1 Architecture and Component Decomposition

As illustrated in Figure 2, the conceptual authentication pipeline decouples request origination from on-chain broadcast across four components: (1) Withdrawal Service (Frontend): Formats the withdrawal request: blockchain network, recipient address, withdrawal amount (in exact integer atomic units, e.g., wei), a UUIDv4 nonce, and a UNIX epoch timestamp. (2) RSA Signing Component: Conceptually hosted within a secure service boundary, this component formats the payload using deterministic JSON serialization, computes the SHA-256 digest, and signs the request using the exchange private key via RSA-PSS. (3) Verification Gateway: An edge gateway that receives the payload and signature, retrieves the authorized RSA public key, reconstructs the message digest, and validates the RSA-PSS signature. (4) Request Validation Engine: Enforces stateful application rules: checks timestamp freshness (±300s window) and verifies nonce uniqueness against a replay cache. Crucially, the signature authenticates the integrity of the request within the internal service boundary, establishing that the payload originated from an authorized signing component.

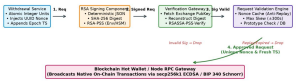


Figure 2. Proposed off-chain cryptocurrency withdrawal authentication architecture using RSA-PSS.

4.2 Deterministic JSON Serialization and Pipeline Algorithm

Cryptographic signature verification fails if distributed services serialize JSON whitespace or key ordering inconsistently. Our system implements deterministic JSON serialization by lexicographically sorting keys, eliminating non-essential whitespace, and encoding to UTF-8. While inspired by canonical formatting, this custom serialization does not implement full RFC 8785 (which requires ECMAScript number formatting and UTF-16 code unit ordering); it provides deterministic byte-level consistency for the defined schema. Per RFC 8017 Section 9.1, EMSA-PSS encoding specifies emBits = modBits − 1 so that integer representative m = OS2IP(EM) satisfies m < n. For a 2048-bit modulus, emBits = 2047 bits (emLen = 256 bytes), with salt length constrained by sLen <= emLen − hLen − 2 (sLen <= 222 bytes for SHA-256; our pipeline uses sLen = 32 bytes). Algorithm 1 details the complete verification lifecycle.

```
Algorithm 1: Off-Chain Withdrawal Authentication Pipeline (Trust Boundaries)
// [Boundary 1: Internal Secure Signer Service]
Input:  Request R = {id, net, addr, amt_atomic, nonce, ts}, Signer Private Key SK_rsa (HSM-isolated)
1:  M_canon ← DeterministicJSON(R)                        // Canonical JSON (RFC 8785 [24] principles)
2:  EM ← EMSA-PSS-ENCODE(M_canon, emBits=modBits-1, sLen=32) // RFC 8017 Section 9.1 encoding
3:  σ ← RSASP1(SK_rsa, OS2IP(EM))                         // Private exponentiation: s = m^d mod n
// [Boundary 2: Network Transit] -> Transmit payload {R, σ} to Verification Edge Gateway
// [Boundary 3: Edge Verification Gateway & Application State Engine]
Input:  Received Payload {R, σ}, Trusted Public Key PK_rsa (retrieved via authenticated PKI)
4:  M_canon' ← DeterministicJSON(R)                       // Re-serialize received request
5:  valid_sig ← RSASSA-PSS-Verify(PK_rsa, M_canon', σ, SHA-256, sLen=32) // Full EMSA-PSS verify
6:  if ¬ valid_sig then return REJECT_SIG                  // Integrity failure or forged signature
7:  if |CurrentTime() - R.ts| > Δ_skew then return REJECT_EXPIRED // Freshness window violation
8:  if R.nonce ∈ NonceStore then return REJECT_REPLAY      // Stateful replay detected
9:  NonceStore.Add(R.nonce); return ACCEPT                 // Authorize dispatch to hot wallet
```

4.3 Prototype Scope and Vital Security Considerations

To ensure academic integrity, the educational prototype must be clearly distinguished from production exchange requirements: (1) Address Validation: The prototype performs an elementary sanity check (verifying that the recipient address string is non-empty and has a minimum length of 10 characters). It does not validate a 0x prefix, hexadecimal encoding, Ethereum EIP-55 checksums, or Bitcoin Bech32/Bech32m formats; production systems require full chain-specific address decoding and checksum verification. (2) Public-Key Management: The prototype generates an RSA-2048 keypair in memory and passes the public key directly to the verifier. In production deployments of this architecture, public-key distribution requires PKI path validation, certificate revocation mechanisms (CRL/OCSP), and periodic key rotation. (3) Private-Key Custody: The prototype stores signing keys in volatile memory for demonstration; production deployments typically protect signing keys using Hardware Security Modules (HSMs) implemented in cryptographic modules validated under FIPS 140-3 [25], where applicable, or secure hardware enclaves based on institutional threat models. (4) Stateful Replay Prevention: The prototype tracks nonces in volatile memory; production deployments with multiple gateway instances typically need durable, shared, atomic nonce check-and-insert mechanisms (e.g., Redis SETNX or SQL unique constraint transactions) with TTL expiration to prevent replay across distributed instances. (5) Identity and Authorization Boundary: The RSA signature verifies only that the request was signed by the holder of the private key and that the payload was not modified in transit. Customer authentication (via MFA and OpenID Connect [26]), withdrawal authorization, ledger balance checks, and AML compliance remain distinct application-tier responsibilities.

5. Experimental Results and Analysis

The system was evaluated using Python 3.15 and cryptography v50.0.2 (OpenSSL 4.0.3) on an AMD Ryzen 5 7520U processor (4 cores, 8 threads @ 2.80 GHz) running Windows 11. High-resolution timings were captured using time.perf_counter_ns() across 10 key-generation trials and 100 signature generation and verification trials per algorithm (840 raw measurements total). All RSA key sizes (2048, 3072, 4096 bits) and ECDSA (secp256k1) were evaluated under identical conditions.

5.1 Adversarial Security Testing Scenarios

Execution of rsa_pss_experiment.py confirmed all five target operational security scenarios: (1) Scenario 1 (Authentic Request): Valid request of 12.5 ETH, represented as an integer amount in wei (12,500,000,000,000,000,000 wei) -> ACCEPTED; (2) Scenario 2 (Tampered Amount): Payload altered to 50.0 ETH; signature verification returned False -> REJECTED; (3) Scenario 3 (Tampered Recipient): Recipient address altered; signature verification failed -> REJECTED; (4) Scenario 4 (Rogue Public Key): Signature evaluated against an unauthorized RSA public key -> REJECTED; (5) Scenario 5 (Replay Attack): Resubmitted request passed cryptographic signature verification, but duplicate nonce was intercepted -> REJECTED BY APPLICATION STATE ENGINE.

5.2 Bundled Wycheproof Test Suite Validation

To evaluate implementation behavior against established test vectors, the test harness executed the bundled RSA-PSS 2048-bit SHA-256/MGF1-SHA256 Wycheproof test set (version 0.9 snapshot, 108 test cases) [14]. The verification engine demonstrated 100% agreement with this selected test suite (63 valid signatures accepted, 45 invalid signatures rejected). While passing these 108 test vectors confirms correct processing of the evaluated cases, it does not constitute full RFC 8017 formal certification or establish the complete absence of implementation vulnerabilities.

5.3 Empirical Benchmarking and Performance Analysis

Table 2 reports empirical benchmark measurements comparing RSA configurations with ECDSA (secp256k1).

Table 2. Empirical benchmark measurements (AMD Ryzen 5 7520U, Python 3.15, OpenSSL 4.0.3, median of 10 keygen / 100 crypto trials).

Algorithm	Security Level	Keygen Median	Sign Median	Verify Median	Sig Size	Internal Asymmetry
RSA-2048	112 bits [5]	50.0708 ms	0.9005 ms	0.0462 ms	256 B	19.49x (Sign / Verify)
RSA-3072	128 bits [5]	253.9132 ms	2.7834 ms	0.0756 ms	384 B	36.82x (Sign / Verify)
RSA-4096	152 bits [5]	633.0616 ms	6.5425 ms	0.115 ms	512 B	56.89x (Sign / Verify)
ECDSA-secp256k1	~128 bits [6]	0.7158 ms	0.7196 ms	0.6214 ms	71 B (DER sample)	1.16x (Sign / Verify)

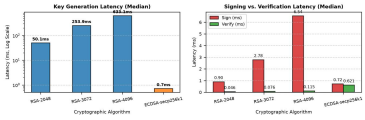


Figure 3. Empirical median latency comparison for RSA and ECDSA key generation, signing, and verification.

The empirical measurements demonstrate three decisive performance characteristics:

1. RSA Asymmetric Cost Structure: RSA displays a pronounced internal asymmetry: verification is 19.49x (RSA-2048) to 56.89x (RSA-4096) faster than its own signing operation. Because public exponent e = 65,537 = 2^{16} + 1 contains only two non-zero bits, straightforward square-and-multiply exponentiation executes in 16 squarings and 1 multiplication (17 operations total), whereas private exponent d requires full modular exponentiation.

2. Comparison with ECDSA: Comparing equivalent 128-bit classical security levels (RSA-3072 vs. ECDSA-secp256k1), ECDSA signing is 3.87x faster (0.7196 ms vs 2.7834 ms), and ECDSA key generation is over 354.7x faster (0.7158 ms vs 253.9132 ms). Conversely, RSA-3072 signature verification (0.0756 ms) is 8.22x faster than ECDSA verification (0.6214 ms). While RSA-2048 verification (0.0462 ms) is 13.45x faster and RSA-4096 (0.115 ms) is 5.4x faster than ECDSA, these represent 112-bit and 152-bit security levels. The recorded ECDSA DER signature sample was 71 bytes, generally varying between 70 and 72 bytes due to variable-length ASN.1 INTEGER encoding.

3. Isolated Latency vs Gateway Throughput: At 0.0756 ms per verification, a single CPU core can theoretically evaluate ~13,227 RSA-3072 signatures per second (vs ~1,609 for ECDSA). These figures represent isolated cryptographic operation timings, not an end-to-end exchange gateway throughput benchmark; real-world system throughput is bounded by network I/O, database persistence, and TLS handshakes.

6. Conclusion

This research examined the cryptographic foundations, operational trade-offs, and practical deployment of RSA in modern security architectures. While elliptic-curve cryptography (ECDSA and Schnorr) rightfully dominates public blockchain consensus layers due to compact public keys and signatures that minimize transaction virtual size (reducing Bitcoin miner fees) and storage serialization (reducing Ethereum transaction gas overhead), RSA serves distinct architectural domains. RSA remains a supported cryptographic option in enterprise PKI and can be deployed in selected off-chain authentication systems, particularly where legacy interoperability and asymmetric verification throughput are advantageous. Our educational prototype confirmed that RSA-PSS with SHA-256 provides verifiable message integrity with rapid gateway verification. However, cryptographic digital signatures alone do not guarantee complete system security; robust authorization requires defense-in-depth across deterministic serialization, stateful replay prevention, secure key custody (HSM storage), and transport encryption (TLS 1.3).

At an approximately equivalent 128-bit classical security strength, RSA-3072 provides robust resistance against conventional mathematical cryptanalysis, though it incurs substantially larger key and signature footprints than 256-bit elliptic curves. Looking toward future cryptographic horizons, RSA-PSS is vulnerable to polynomial-time integer factorization via Shor's algorithm on fault-tolerant quantum computers, as is standard elliptic-curve cryptography under discrete logarithm quantum attacks. Consequently, organizations deploying RSA-3072 today must maintain crypto-agility and prepare migration pathways toward standardized post-quantum digital signature algorithms, including NIST FIPS 204 (ML-DSA) [20] and FIPS 205 (SLH-DSA) [21].

7. References

[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," Communications of the ACM, vol. 21, no. 2, pp. 120–126, Feb. 1978 (MIT/LCS/TM-82, Apr. 1977). https://doi.org/10.1145/359340.359342

[2] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, IETF, Nov. 2016. https://www.rfc-editor.org/rfc/rfc8017

[3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, IETF, July 2026. https://www.rfc-editor.org/rfc/rfc9846

[4] National Institute of Standards and Technology (NIST), "Digital Signature Standard (DSS)," Federal Information Processing Standards Publication (FIPS) 186-5, Feb. 2023. https://doi.org/10.6028/NIST.FIPS.186-5

[5] E. Barker, L. Chen, A. Roginsky, A. Vassilev, R. Davis, and S. Simon, "Recommendation for Pair-Wise Key-Establishment Schemes Using Integer Factorization Cryptography," NIST Special Publication (SP) 800-56B Rev. 2, Mar. 2019. https://doi.org/10.6028/NIST.SP.800-56Br2

[6] E. Barker, "Recommendation for Key Management: Part 1 – General," NIST Special Publication (SP) 800-57 Part 1 Rev. 5, May 2020. https://doi.org/10.6028/NIST.SP.800-57pt1r5

[7] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of eliminating errors in cryptographic computations," Journal of Cryptology, vol. 14, no. 2, pp. 101–119, 2001. https://doi.org/10.1007/s001450010016

[8] P. Wuille, J. Nick, and T. Ruffing, "Schnorr Signatures for secp256k1," Bitcoin Improvement Proposal (BIP) 340, Jan. 2020. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

[9] E. Lombrozo, J. Lau, and P. Wuille, "Segregated Witness (Consensus layer)," Bitcoin Improvement Proposal (BIP) 141, Dec. 2015. https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki

[10] J. Nick, T. Ruffing, and E. Jin, "MuSig2 for BIP340-compatible Multi-Signatures," Bitcoin Improvement Proposal (BIP) 327, 2023. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki

[11] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, Jan. 2017. https://www.rfc-editor.org/rfc/rfc8032

[12] Bitcoin Core Developers, "Bitcoin Developer Guide: Transactions," Bitcoin Project, n.d. [Online; accessed 2026]. https://developer.bitcoin.org/devguide/transactions.html

[13] Ethereum Foundation, "Ethereum Developer Documentation: Transactions and Cryptographic Signatures," n.d. [Online; accessed 2026]. https://ethereum.org/en/developers/docs/transactions/

[14] Google Security Research, "Project Wycheproof: Cryptographic Test Vectors for Asymmetric Algorithms," GitHub repository (v0.9 snapshot), n.d. [Online; accessed 2026]. https://github.com/google/wycheproof

[15] V. Shoup, "OAEP Reconsidered," Journal of Cryptology, vol. 15, no. 4, pp. 223–249, Sep. 2002 (extended abstract in Advances in Cryptology – CRYPTO 2001, LNCS 2139, pp. 239–259, Springer). https://doi.org/10.1007/s00145-002-0133-9

[16] E. Fujisaki, T. Okamoto, D. Pointcheval, and J. Stern, "RSA-OAEP is Secure under the RSA Assumption," Journal of Cryptology, vol. 17, no. 2, pp. 81–104, Mar. 2004 (extended abstract in Advances in Cryptology – CRYPTO 2001, LNCS 2139, pp. 260–274, Springer). https://doi.org/10.1007/s00145-002-0204-y

[17] M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption," in Advances in Cryptology – EUROCRYPT '94, Lecture Notes in Computer Science, vol. 950, pp. 92–111, Springer, 1995. https://doi.org/10.1007/BFb0053428

[18] D. Benjamin and A. Popov, "Legacy RSASSA-PKCS1-v1_5 Code Points for TLS 1.3," RFC 9963, IETF, April 2026. https://doi.org/10.17487/RFC9963

[19] K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan, and D. Stebila, "Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3," RFC 10024, IETF, August 2026. https://doi.org/10.17487/RFC10024

[20] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," FIPS 204, August 2024. https://doi.org/10.6028/NIST.FIPS.204

[21] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard," FIPS 205, August 2024. https://doi.org/10.6028/NIST.FIPS.205

[22] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, November 2007. https://doi.org/10.6028/NIST.SP.800-38D

[23] M. Bellare and P. Rogaway, "The Exact Security of Digital Signatures—How to Sign with RSA and Rabin," in Advances in Cryptology – EUROCRYPT '96, Lecture Notes in Computer Science, vol. 1070, pp. 399–416, Springer, 1996. https://doi.org/10.1007/3-540-68339-9_34

[24] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, IETF, June 2020. https://doi.org/10.17487/RFC8785

[25] National Institute of Standards and Technology, "Security Requirements for Cryptographic Modules," FIPS 140-3, March 2019. https://doi.org/10.6028/NIST.FIPS.140-3

[26] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1," OpenID Foundation, Nov. 2014. https://openid.net/specs/openid-connect-core-1_0.html

[27] P. Wuille, J. Nick, and A. Towns, "Taproot: SegWit version 1 spending rules," Bitcoin Improvement Proposal (BIP) 341, Jan. 2020. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki

[28] R. Moore, "ERC-2098: Compact Signature Representation," Ethereum Improvement Proposals, March 2019. https://eips.ethereum.org/EIPS/eip-2098

[29] V. Buterin and M. H. Swende, "EIP-198: Big Integer Modular Exponentiation Precompile," Ethereum Improvement Proposals, March 2017. https://eips.ethereum.org/EIPS/eip-198